```
┌──────────── ○○○ ────────────┐
│ Semgrep CLI                 │
└─────────────────────────────┘

⋮ Loading rules from registry
Scanning 12 files (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✓ Semgrep Code (SAST)
  ✓ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

━━━
```

```
197 ¦ body: bot.training.state ? bot.greet(`${user.id}`) :
    `${config.get<string>('application.chatBot.name')} isn't ready at the moment, please wait
    while I set things up`
```

fileServer.ts

❯❯ javascript.express.security.audit.express-res-sendfile.express-res-sendfile

❮❮ Blocking ❯❯

The application processes user-input, this is passed to res.sendFile which can allow an attacker to arbitrarily read files on the system through path traversal. It is recommended to perform input validation in addition to canonicalizing the path. This allows you to validate the path against the intended directory it should be accessing.
Details: https://sg.run/7DJk

```
33 ¦ res.sendFile(path.resolve('ftp/', file))
```

fileUpload.ts

❯❯ javascript.express.security.audit.express-libxml-vm-noent.express-libxml-vm-noent

❮❮ Blocking ❯❯

Detected use of parseXml() function with the `noent` field set to `true`. This can lead to an XML External Entities (XXE) attack if untrusted data is passed into it.
Details: https://sg.run/n8Ag

```
83 ¦ const xmlDoc = vm.runInContext('libxml.parseXml(data, { noblanks: true, noent: true,
    nocdata: true })', sandbox, { timeout: 2000 })
```

login.ts

❯❯❯❯ javascript.express.db.sequelize-express.sequelize-express

❮❮ Blocking ❯❯

Untrusted input might be used to build a database query, which can lead to a SQL injection vulnerability. An attacker can execute malicious SQL statements and gain unauthorized access to sensitive data, modify, delete data, or execute arbitrary system commands. To prevent this vulnerability, use prepared statements that do not concatenate user-controllable strings and use parameterized queries where SQL commands and user data are strictly separated. Also, consider using an object-relational (ORM) framework to operate with safer abstractions.
Details: https://sg.run/6JKdw

```
34 ¦ models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
    password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
    UserModel, plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge
    loginBenderChallenge loginJimChallenge
```

❯❯❯ javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection

❮❮ Blocking ❯❯

Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it is recommended to use parameterized queries or prepared statements.
Details: https://sg.run/gjoe

```
34 ¦ models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
    password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
    UserModel, plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge
    loginBenderChallenge loginJimChallenge
```

redirect.ts

❯❯ javascript.express.open-redirect-deepsemgrep.open-redirect-deepsemgrep

❮❮ Blocking ❯❯

The application builds a URL using user-controlled input which can lead to an open redirect vulnerability. An attacker can manipulate the URL and redirect users to an arbitrary domain. Open redirect vulnerabilities can lead to issues such as Cross-site scripting (XSS) or redirecting to a malicious domain for activities such as phishing to capture users' credentials. To prevent this vulnerability perform strict input validation of the domain against an allowlist of approved domains. Notify a user in your application that they are leaving the website. Display a domain where they are redirected to the user. A user can then either accept or deny the redirect to an untrusted site.
Details: https://sg.run/BDbW

```
19 ┆ res.redirect(toUrl)
```

❯❯ javascript.express.security.audit.express-open-redirect.express-open-redirect

❮❮ Blocking ❯❯

The application redirects to a URL specified by user-supplied input `query` that is not validated. This could redirect users to malicious locations. Consider using an allow-list approach to validate URLs, or warn users they are being redirected to a third-party website.
Details: https://sg.run/EpoP

```
19 ┆ res.redirect(toUrl)
```

❯❯ javascript.express.web.tainted-redirect-express.tainted-redirect-express

❮❮ Blocking ❯❯

The application builds a URL using user-controlled input which can lead to an open redirect vulnerability. An attacker can manipulate the URL and redirect users to an arbitrary domain. Open redirect vulnerabilities can lead to issues such as Cross-site scripting (XSS) or redirecting to a malicious domain for activities such as phishing to capture users' credentials. To prevent this vulnerability perform strict input validation of the domain against an allowlist of approved domains. Notify a user in your application that they are leaving the website. Display a domain where they are redirected to the user. A user can then either accept or deny the redirect to an untrusted site.
Details: https://sg.run/7KEAw

```
19 ┆ res.redirect(toUrl)
```

showProductReviews.ts

❯❯❯❯ javascript.express.mongodb.express-mongo-nosqli.express-mongo-nosqli

❮❮ Blocking ❯❯

Detected a `../data/mongodb` statement that comes from a `req` argument. This could lead to NoSQL injection if the variable is user-controlled and is not properly sanitized. Be sure to properly sanitize the data if you absolutely must pass request data into a mongo query.
Details: https://sg.run/Qqxq

```
36 ┆ db.reviewsCollection.find({ $where: 'this.product == ' + id }).then((reviews: Review[])
     => {
```

updateUserProfile.ts

❯❯ javascript.express.web.cookies-default-express.cookies-default-express

❮❮ Blocking ❯❯
Having default cookie settings is insecure because they often lack critical security attributes, leaving cookies vulnerable to various attacks like CSRF or XSS. Always configure cookies with security attributes: `HttpOnly`, `Secure`, `SameSite`.
Details: https://sg.run/JDG5Z

```
40 ┆ res.cookie('token', updatedToken)
```

❯❯ javascript.express.session-fixation.session-fixation
❮❮ Blocking ❯❯
Detected `req` argument which enters `res.cookie`, this can lead to session fixation vulnerabilities if an attacker can control the cookie value. This vulnerability can lead to unauthorized access to accounts, and in some esoteric cases, Cross-Site-Scripting (XSS). Users should not be able to influence cookies directly, for session cookies, they should be generated securely using an approved session management library. If the cookie does need to be set by a user, consider using an allow-list based approach to restrict the cookies which can be set.
Details: https://sg.run/oqDv

```
40 ┆ res.cookie('token', updatedToken)
```

userProfile.ts
❯❯ javascript.browser.security.eval-detected.eval-detected
❮❮ Blocking ❯❯
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this content can be input from outside the program, this may be a code injection vulnerability. Ensure evaluated content is not definable by external sources.
Details: https://sg.run/7ope

```
62 ┆ username = eval(code) // eslint-disable-line no-eval
```

❯❯❯ javascript.lang.security.audit.code-string-concat.code-string-concat
❮❮ Blocking ❯❯
Found data from an Express or Next web request flowing to `eval`. If this data is user-controllable this can lead to execution of arbitrary system commands in the context of your application process. Avoid `eval` whenever possible.
Details: https://sg.run/96Yk

```
62 ┆ username = eval(code) // eslint-disable-line no-eval
```

❯❯❯❯ javascript.express.code.eval-express.eval-express
❮❮ Blocking ❯❯
The application might dynamically evaluate untrusted input, which can lead to a code injection vulnerability. An attacker can execute arbitrary code, potentially gaining complete control of the system. To prevent this vulnerability, avoid executing code containing user input. If this is unavoidable, validate and sanitize the input, and use safe alternatives for evaluating user input.
Details: https://sg.run/PeoWE

```
62 ┆ username = eval(code) // eslint-disable-line no-eval
```

videoHandler.ts
❯❯ javascript.lang.security.audit.unknown-value-with-script-tag.unknown-value-with-script-tag
❮❮ Blocking ❯❯

Cannot determine what 'subs' is and it is used with a '<script>' tag. This could be susceptible to cross-site scripting (XSS). Ensure 'subs' is not externally controlled, or sanitize this data.
Details: https://sg.run/1Zy1

```
58 ┆ challengeUtils.solveIf(challenges.videoXssChallenge, () => { return utils.contains(subs,
   '</script><script>alert(`xss`)</script>') })
 ⋮┆----------------------------------------
71 ┆ compiledTemplate = compiledTemplate.replace('<script id="subtitle"></script>', '<script
   id="subtitle" type="text/vtt" data-label="English" data-lang="en">' + subs + '</script>')
```

```
╭──────────────────────────╮
│ Scan Summary             │
╰──────────────────────────╯
```

✅ Scan completed successfully.
• Findings: 18 (18 blocking)
• Rules run: 385
• Targets scanned: 12
• Parsed lines: ~100.0%
• No ignore information available
Ran 385 rules on 12 files: 18 findings.